Звіт про виконання завдання № 5.4

Обчислення сум за допомогою рекурсії

Варіант 7

Виконав: Джамалов Олександр Олегович, група РІ-11

1. Мета роботи

Навчитися реалізовувати рекурсивні функції, що виконують обчислення на рекурсивному спуску та на рекурсивному підйомі.

2. Умова завдання

За допомогою рекурсивних функцій обчислити значення суми чи добутку.

В одній програмі слід вивести результати звертання до п’яти функцій, які реалізують такі способи:

1) обчислення здійснюються на рекурсивному спуску, значення параметра в рекурсивних викликах – спадає;
2) обчислення здійснюються на рекурсивному спуску, значення параметра в рекурсивних викликах – зростає;
3) обчислення здійснюються на рекурсивному підйомі, значення параметра в рекурсивних викликах – спадає;
4) обчислення здійснюються на рекурсивному підйомі, значення параметра в рекурсивних викликах – зростає;
5) обчислення здійснюється ітераційним способом (для контролю правильності рекурсивних способів).

Всі п’ять результатів мають збігатися. Всю необхідну функціям інформацію слід передавати лише за допомогою параметрів.

Варіант 7.

$$\sum_{k=1}^{N} \frac{1}{(2k+1)^2}$$

Рисунок 1 – Умова завдання 5.4, варіант 7

3. Математична модель

$S = \Sigma_{k=1..N}\, a(k)$, $a(k) = 1 / (2k + 1)^2$. Доданок обчислює окрема функція term(k).

Спуск (5.4.1, 5.4.2): доданок додається до накопиченої суми *перед* рекурсивним викликом, сума передається параметром, базовий випадок повертає готову суму. **Підйом** (5.4.3, 5.4.4): спочатку виконується рекурсивний виклик, а доданок додається *після* повернення з нього; базовий випадок повертає 0.

Параметр спадає: k = N, N − 1, …, 1, базовий випадок k = 0. Параметр зростає: k = 1, 2, …, N, базовий випадок k > N. Кожен рекурсивний спосіб має N + 1 рівень. Для захисту стека прийнято $1 \le N \le 1000$. Ітераційний спосіб 5.4.5 використано для контролю.

4. Структура програми

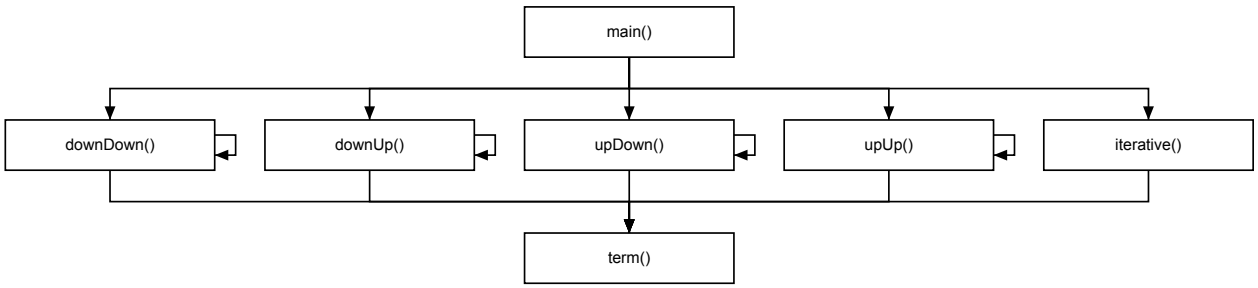


Рисунок 2 – Структурна схема програми (стрілка праворуч – рекурсивний виклик)

Таблиця 1 – Специфікація функцій програми

№	Функція	Спосіб	Параметри	Результат
–	double term(int k)	доданок 1/(2k+1)²	k – номер доданка	a(k)
5.4.1	double downDown(int k, double sum)	спуск, параметр спадає	k – номер; sum – накопичена сума	S
5.4.2	double downUp(int k, int n, double sum)	спуск, параметр зростає	k; n = N; sum	S
5.4.3	double upDown(int k)	підйом, параметр спадає	k – номер доданка	S
5.4.4	double upUp(int k, int n)	підйом, параметр зростає	k; n = N	S
5.4.5	double iterative(int n)	ітераційний (контроль)	n = N	S
–	int main()	введення N, виклик 5 способів, таблиця	–	код завершення

Таблиця 2 – Змінні програми

Ім'я	Тип	Функція	Призначення
n	int	main, downUp, upUp, iterative	кількість доданків N
k	int	term та рекурсивні функції	номер поточного доданка (параметр рекурсії)
sum	double	downDown, downUp, iterative	накопичена сума
rest	double	upDown, upUp	сума, повернена рекурсивним викликом
d	double	term	2k + 1
s1 … s5	double	main	результати п'яти способів

5. Алгоритми програми

Блок-схеми виконано за ГОСТ 19.701-90 (ДСТУ ISO 5807:2016): «Початок» і «Кінець» – символ «термінатор», введення/виведення – «дані» (паралелограм), обчислення – «процес», виклик функції – «зумовлений процес», умова – «рішення». Лінії потоку, що йдуть зверху вниз і зліва направо, стрілками не позначено; стрілку показано лише на лінії повернення (повторення циклу). Рекурсивний виклик зображено символом «зумовлений процес».

UML-діаграми діяльності (UML 2.5) виконано в рамці «act»: початковий вузол – зафарбоване коло, кінцевий – коло з точкою, дії – прямокутники із заокругленими кутами (дія-виклик позначена символом «вила»), розгалуження та злиття – ромби, умови переходів – сторожові вирази у квадратних дужках. Усі переходи позначено стрілками.

5.1. Головна функція main()

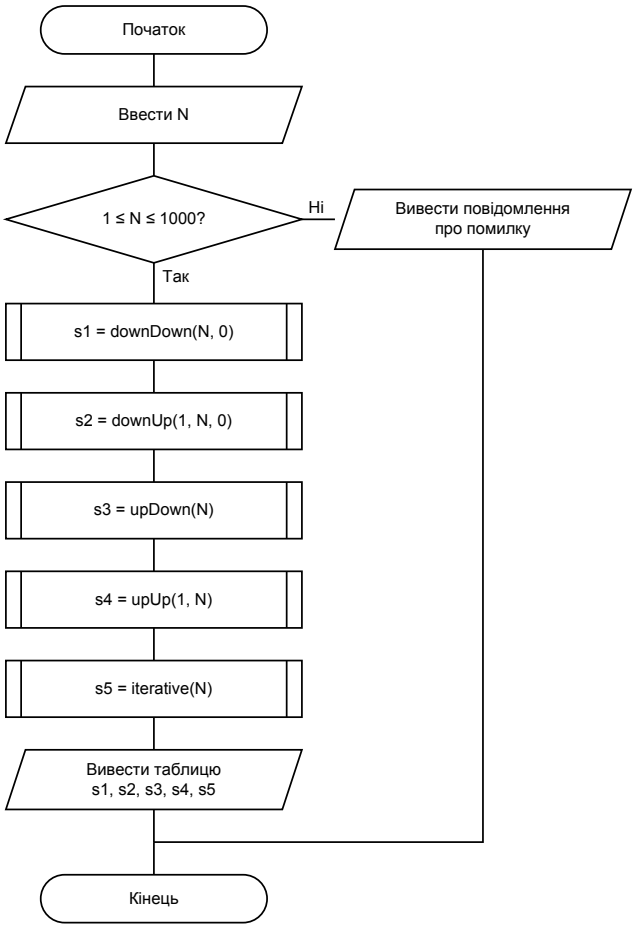


Рисунок 3 – Блок-схема функції main()

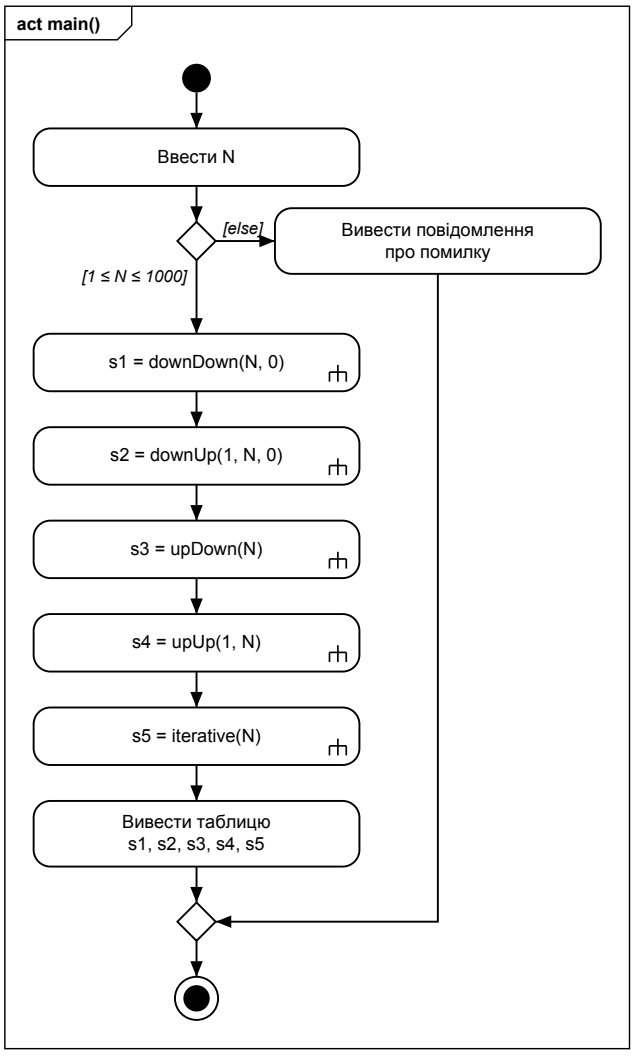


Рисунок 4 – UML-діаграма діяльності функції main()

5.2. Функція доданка term(k)

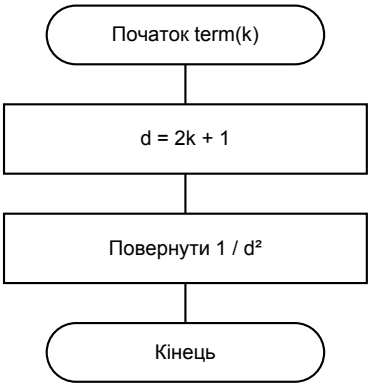


Рисунок 5 – Блок-схема функції term(k)

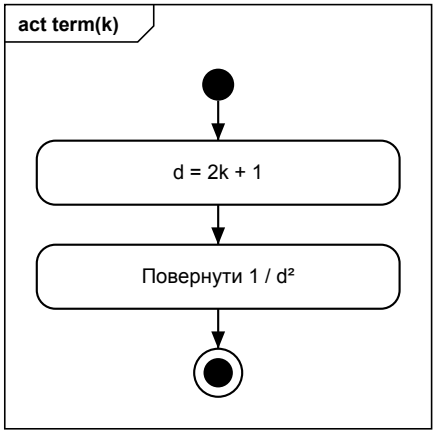


Рисунок 6 – UML-діаграма діяльності функції term(k)

5.3. Спосіб 5.4.1 – рекурсивний спуск, параметр спадає: downDown(k, sum)

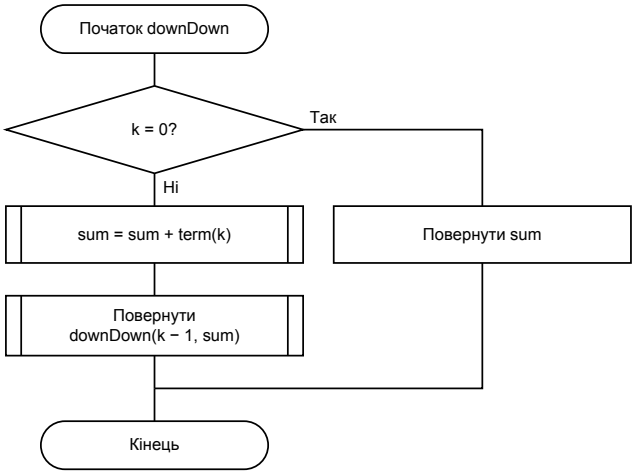


Рисунок 7 – Блок-схема функції downDown(k, sum)

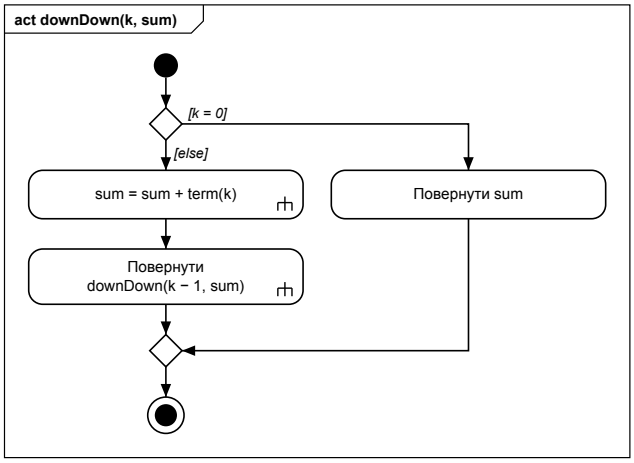


Рисунок 8 – UML-діаграма діяльності функції downDown(k, sum)

5.4. Спосіб 5.4.2 – рекурсивний спуск, параметр зростає: downUp(k, n, sum)

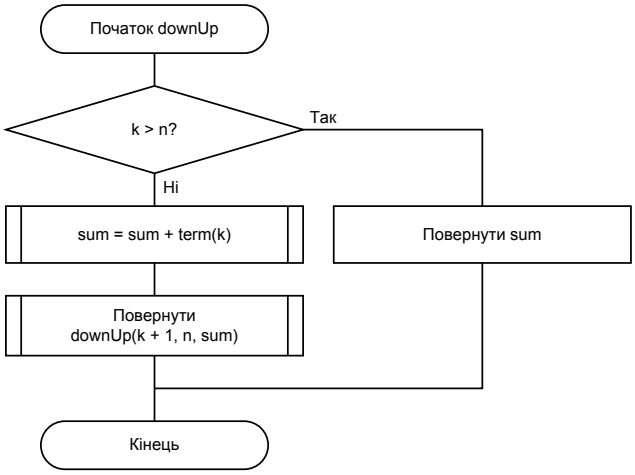


Рисунок 9 – Блок-схема функції downUp(k, n, sum)

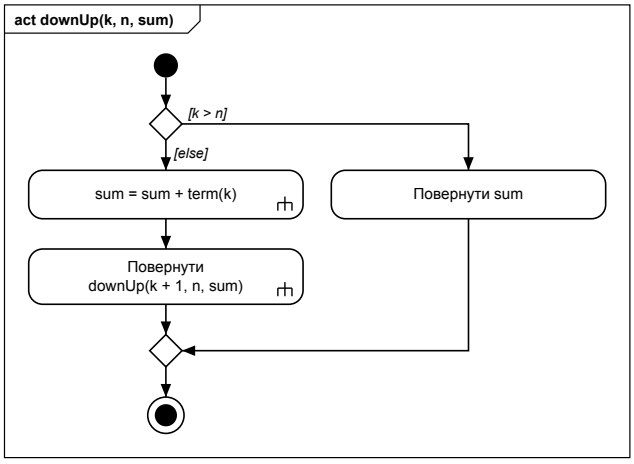


Рисунок 10 – UML-діаграма діяльності функції downUp(k, n, sum)

5.5. Спосіб 5.4.3 – рекурсивний підйом, параметр спадає: upDown(k)

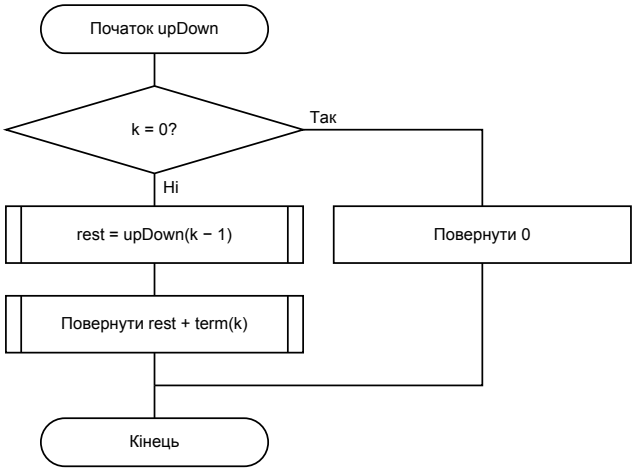


Рисунок 11 – Блок-схема функції upDown(k)

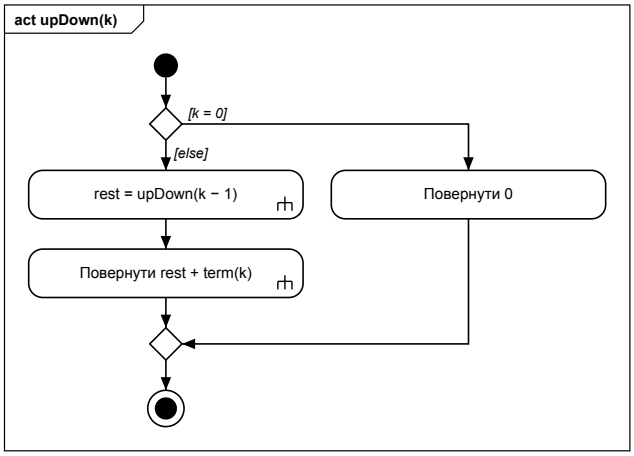


Рисунок 12 – UML-діаграма діяльності функції upDown(k)

5.6. Спосіб 5.4.4 – рекурсивний підйом, параметр зростає: upUp(k, n)

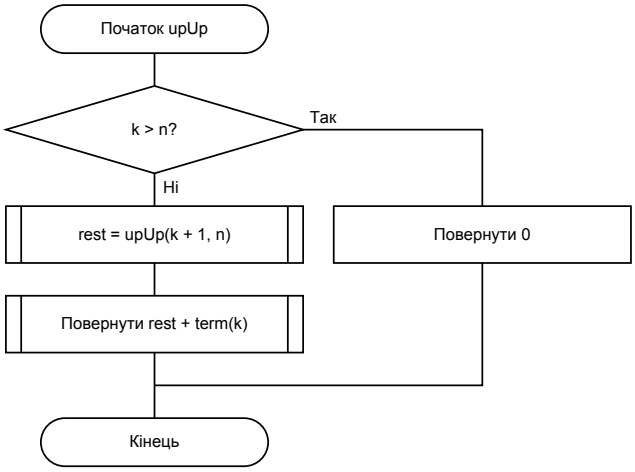


Рисунок 13 – Блок-схема функції upUp(k, n)

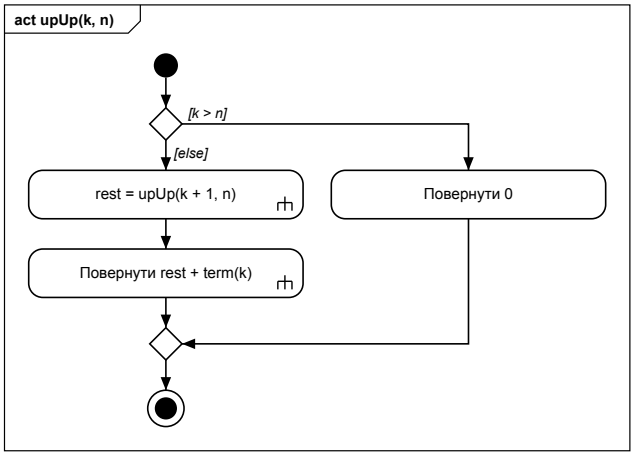


Рисунок 14 – UML-діаграма діяльності функції upUp(k, n)

5.7. Спосіб 5.4.5 – ітераційний: iterative(n)

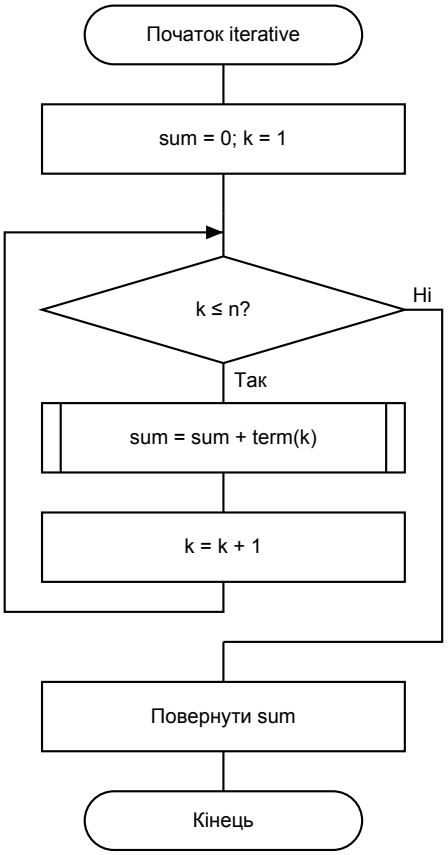


Рисунок 15 – Блок-схема функції iterative(n)

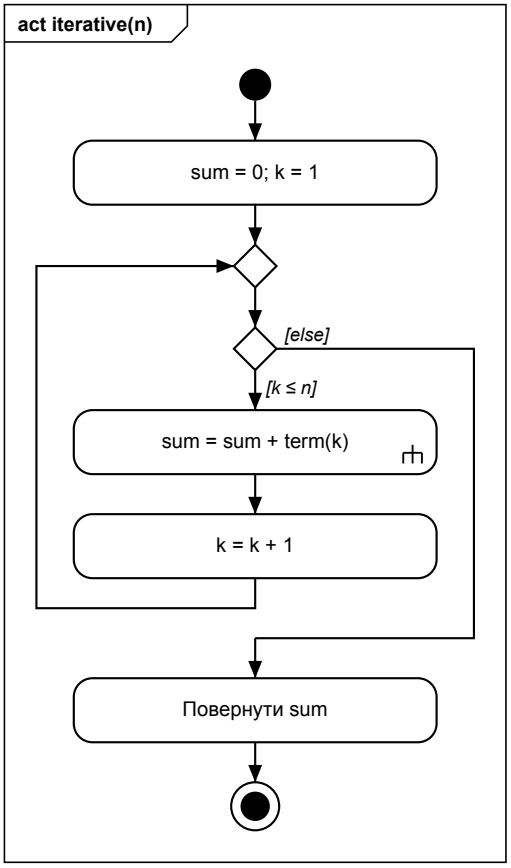


Рисунок 16 – UML-діаграма діяльності функції iterative(n)

6. Текст програми

Кожна дія програми супроводжується коментарем. Головну функцію main() вимкнено директивою #ifndef UNIT_TEST, щоб файл можна було підключити до тестів.

Файл `lab_5_4.cpp`

```
// Лабораторна робота 5.4, варіант 7. Джамалов О. О., PI-11.
// S = sum_{k=1..N} 1 / (2k + 1)^2 п'ятьма способами.
#include <iomanip>  // setw, setprecision, fixed
#include <iostream> // cin, cout, cerr
using namespace std;

// Доданок суми a(k) = 1 / (2k + 1)^2.
double term(int k)
{
    const double d = 2.0 * k + 1; // знаменник без квадрата
    return 1 / (d * d);           // значення доданка
}

// 5.4.1: обчислення на рекурсивному спуску, параметр спадає (k = N ... 1).
double downDown(int k, double sum)
{
    if (k == 0)                   // базовий випадок: усі доданки враховано
        return sum;               // повертаємо накопичену суму
    sum += term(k);               // додаємо доданок до виклику
    return downDown(k - 1, sum);  // рекурсивний виклик з меншим k
}

// 5.4.2: обчислення на рекурсивному спуску, параметр зростає (k = 1 ... N).
double downUp(int k, int n, double sum)
{
    if (k > n)                    // базовий випадок: k вийшло за N
        return sum;               // повертаємо накопичену суму
    sum += term(k);               // додаємо доданок до виклику
    return downUp(k + 1, n, sum); // рекурсивний виклик з більшим k
}
// 5.4.3: обчислення на рекурсивному підйомі, параметр спадає (k = N ... 1).
double upDown(int k)
{
    if (k == 0)                   // базовий випадок: порожня сума
        return 0;                 // сума нуля доданків
    const double rest = upDown(k - 1); // спочатку рекурсивний виклик
    return rest + term(k);        // додавання після повернення
}

// 5.4.4: обчислення на рекурсивному підйомі, параметр зростає (k = 1 ... N).
double upUp(int k, int n)
{
    if (k > n)                    // базовий випадок: k вийшло за N
        return 0;                 // сума нуля доданків
    const double rest = upUp(k + 1, n); // спочатку рекурсивний виклик
    return rest + term(k);        // додавання після повернення
}

// 5.4.5: ітераційний спосіб для контролю.
double iterative(int n)
{
    double sum = 0;               // початкове значення суми
    for (int k = 1; k <= n; ++k)  // перебір k = 1 ... N
        sum += term(k);           // додаємо доданок
    return sum;                   // повертаємо суму
}

#ifndef UNIT_TEST
int main()
```

```
{
    int n;                                  // кількість доданків N
    cout << "N (1..1000): ";                // запрошення до введення
    if (!(cin >> n) || n < 1 || n > 1000)   // перевірка введення
    {
        cerr << "Error: 1 <= N <= 1000.\n"; // повідомлення про помилку
        return 1;                           // завершення з кодом помилки
    }

    const double s1 = downDown(n, 0);       // 5.4.1
    const double s2 = downUp(1, n, 0);      // 5.4.2
    const double s3 = upDown(n);            // 5.4.3
    const double s4 = upUp(1, n);           // 5.4.4
    const double s5 = iterative(n);         // 5.4.5
    const char* line = "+-------+------------------------------+-----------------+\n";
    cout << "Lab 5.4 / Variant 7, N = " << n << '\n' << line; // заголовок і верхня межа
    cout << "|   #   | Method                       |       Sum       |\n" << line;
    cout << fixed << setprecision(12);      // формат чисел
    cout << "| 5.4.1 | descent, k decreases         | " << setw(15) << s1 << " |\n";
    cout << "| 5.4.2 | descent, k increases         | " << setw(15) << s2 << " |\n";
    cout << "| 5.4.3 | ascent, k decreases          | " << setw(15) << s3 << " |\n";
    cout << "| 5.4.4 | ascent, k increases          | " << setw(15) << s4 << " |\n";
    cout << "| 5.4.5 | iterative (check)            | " << setw(15) << s5 << " |\n";
    cout << line;                           // нижня межа таблиці
    return 0;                               // успішне завершення
}
#endif
```

7. Автоматизоване тестування

Файл `test_5_4.cpp`

```
// Unit-тести п'яти способів обчислення суми із завдання 5.4.
#define UNIT_TEST       // вимикаємо main() у файлі програми
#include "lab_5_4.cpp"  // підключаємо функції
#include <cmath>        // abs

int main()
{
    int failed = 0;                                     // лічильник невдалих тестів
    for (int n : {1, 2, 5, 20, 100, 1000})              // тестові значення N
    {
        long double expected = 0;                       // еталон з long double
        for (int j = 1; j <= n; ++j)
            expected += 1.0L / ((2.0L * j + 1) * (2.0L * j + 1));
        const double results[] = {downDown(n, 0), downUp(1, n, 0),
            upDown(n), upUp(1, n), iterative(n)};       // усі п'ять способів
        bool ok = true;                                 // ознака успіху
        for (double value : results)                    // порівняння кожного
            ok = ok && std::abs(value - expected) < 1e-12;
        cout << (ok ? "PASS" : "FAIL") << " N=" << n
             << " (all five methods)\n";                // результат тесту
        if (!ok) ++failed;                              // рахуємо помилку
    }
    cout << "5.4: " << 6 - failed << "/6 passed\n";     // підсумок
    return failed ? 1 : 0;                              // код завершення для CTest
}
```

Фактичний вивід тестової програми:

```
PASS N=1 (all five methods)
PASS N=2 (all five methods)
PASS N=5 (all five methods)
PASS N=20 (all five methods)
PASS N=100 (all five methods)
PASS N=1000 (all five methods)
5.4: 6/6 passed
```

Таблиця 3 – Результати unit-тестів

№	N	Перевірка	Результат
1	1	усі 5 способів = еталон (long double), допуск 10^{-12}	пройдено
2	2	усі 5 способів = еталон (long double), допуск 10^{-12}	пройдено
3	5	усі 5 способів = еталон (long double), допуск 10^{-12}	пройдено
4	20	усі 5 способів = еталон (long double), допуск 10^{-12}	пройдено
5	100	усі 5 способів = еталон (long double), допуск 10^{-12}	пройдено
6	1000	усі 5 способів = еталон (long double), допуск 10^{-12}	пройдено

8. Результати виконання програми

Компілятор clang++, стандарт C++17, ключі -Wall -Wextra -Wpedantic (попереджень немає). Вхідні дані: 5.

```
N (1..1000): 5
Lab 5.4 / Variant 7, N = 5
+-------+------------------------------+-----------------+
|   #   | Method                       |       Sum       |
+-------+------------------------------+-----------------+
| 5.4.1 | descent, k decreases         |  0.192129416199 |
| 5.4.2 | descent, k increases         |  0.192129416199 |
| 5.4.3 | ascent, k decreases          |  0.192129416199 |
| 5.4.4 | ascent, k increases          |  0.192129416199 |
| 5.4.5 | iterative (check)            |  0.192129416199 |
+-------+------------------------------+-----------------+
```

9. Посилання на Git-репозиторій

https://github.com/dzhamsasha55-dev/lab-5

Висновки

Реалізовано чотири рекурсивні способи обчислення суми: на рекурсивному спуску (обчислення до виклику, сума передається параметром) і на рекурсивному підйомі (обчислення після повернення), кожен – зі спадним і зростаючим параметром, а також ітераційний спосіб для контролю.

Усі п'ять результатів збігаються (таблиця виводу), що підтверджено 6 unit-тестами для N від 1 до 1000.